11711BCC008
Matheus José da Costa

Primeira Atividade de Buffer Overflow

1) Explique as etapas mais comuns para realizar um ataque buffer overflow bem sucedido.

Dados são enviados de um dispositivo de entrada e enviados para o buffer, que vai para a memória, cujo são processados e então vão para o dispositivo de saída.

Então, a partir de que um texto é gravado na memória, temos o seguinte cenário:

Na memória temos a área de Text, Data, Heap e Stack. Na área de texto é onde contém as instruções do programa. Na área de dados é onde fica as variáveis globais e informações estáticas do programa. No heap é onde se faz a alocação dinâmica de variáveis. E a pilha é o bloco de memória que se armazena as variáveis locais, guarda parâmetros de variáveis e armazena valores de retorno.

Para um ataque ser bem sucedido, precisamos alterar o retorno do meu programa pra região de Stack pointer, pois assim conseguimos gerar um buffer overflow a partir de sobrescrever um espaço de memória que não suporta uma informação tão grande.

A injeção de código diretamente na pilha é a maneira mais fácil de fazer o ataque.

2) Uma vez que a grande parte das linguagens vulneráveis a buffer overflow são derivadas da linguagem C, qual é a razão desse ataque também acontecer em sistemas desenvolvidos com outras linguagens de programação?

Geralmente, linguagens de programação que não fazem checagem de limite ou alteração dinâmica de tamanho do buffer que são frágeis a este problema.

A linguagem C, por exemplo, é uma linguagem exemplo, porque possui funções como o strcpy e afins que não fazem esse tipo de checagem.

Outras linguagens são sim suscetíveis a ataques Buffer overflow, como é o caso do Java, que é uma linguagem derivada da linguagem da C. E o buffer overflow é algo passível de acontecer por conta do tratamento de cada linguagem em específico. Então, por exemplo, se dada linguagem utiliza algum recurso -- estrutura ou até mesmo rotinas da linguagem C -- podem ser atacadas.

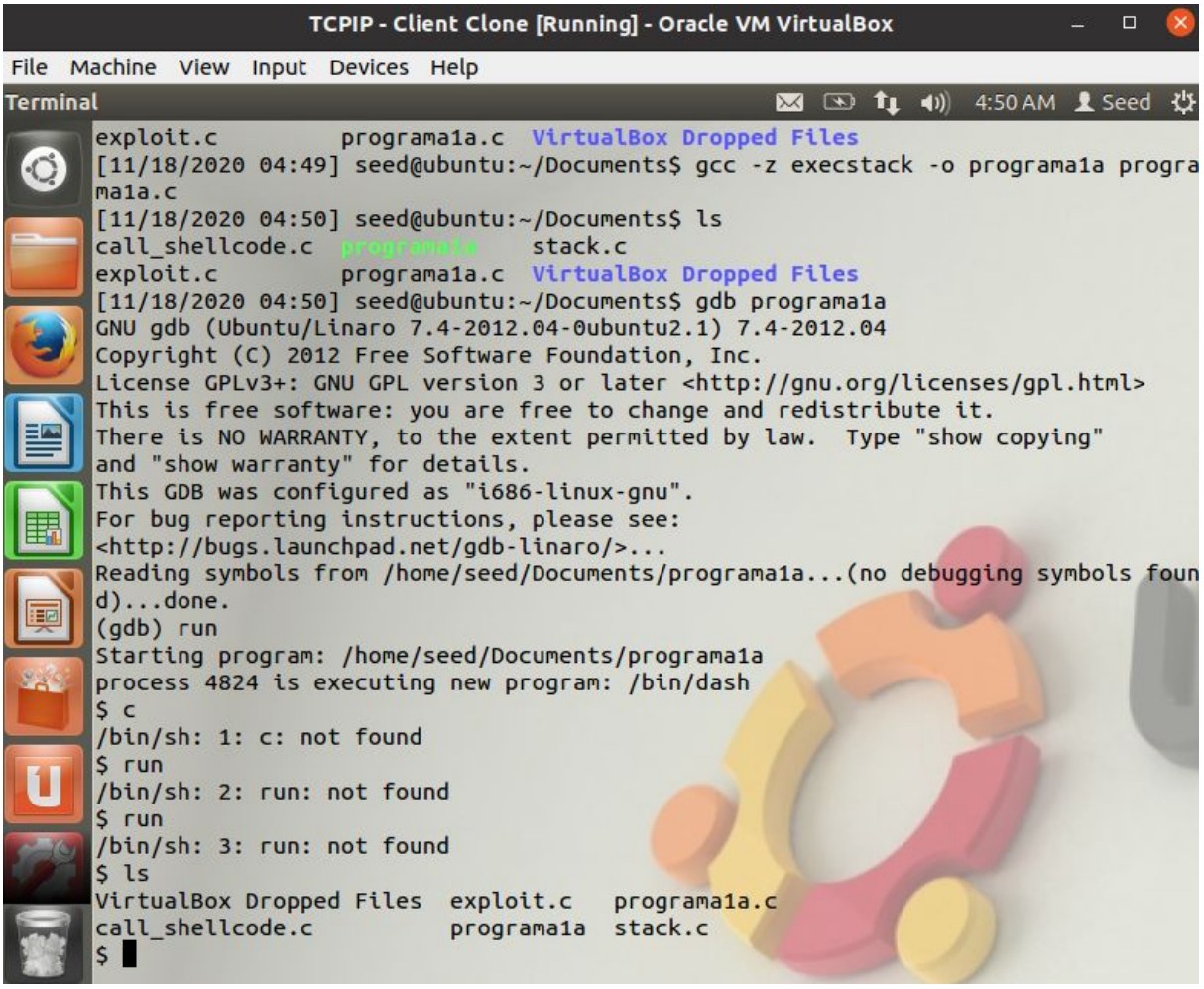
Fontes: retirei as informações da aula de Buffer Overflow.

Segunda Atividade de Buffer Overflow

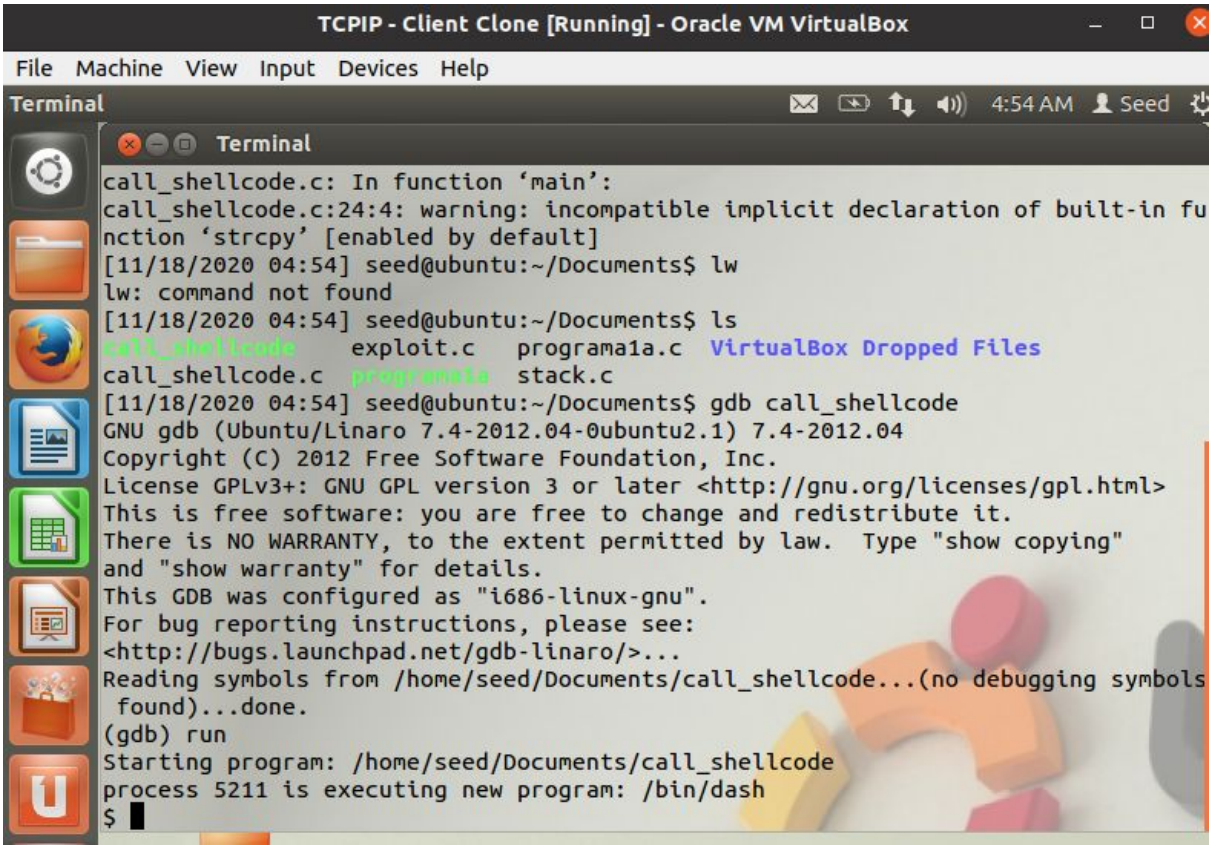
1- Etapa de configuração

2-

a) Sim, abriu uma shell.



b) Sim, foi possível obter uma shell a partir do programa shell_code.c



3)

a) Esse programa basicamente gera um buffer overflow a partir de um arquivo contendo certo tipo de texto. Basicamente, para ser gerado um buffer overflow, é necessário que o arquivo contenha textos de pelo menos 24 caracteres.

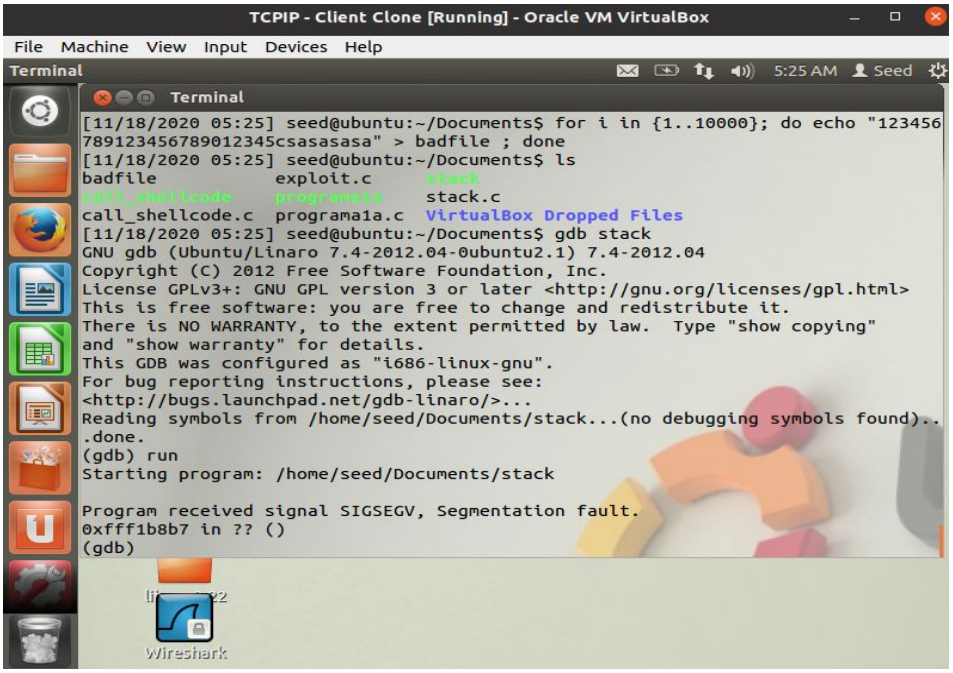
b) O trecho de código vulnerável a buffer overflow está na função bof(char *str), porque faz um copy da string do arquivo pro buffer[24].

c) Ao executar este programa, não foi gravado nenhuma informação no arquivo criado “badfile”.

d) Para esta questão, precisei preencher o arquivo badfile com palavras de pelo menos 24 caracteres.

utilizei o seguinte comando pra preencher o arquivo:

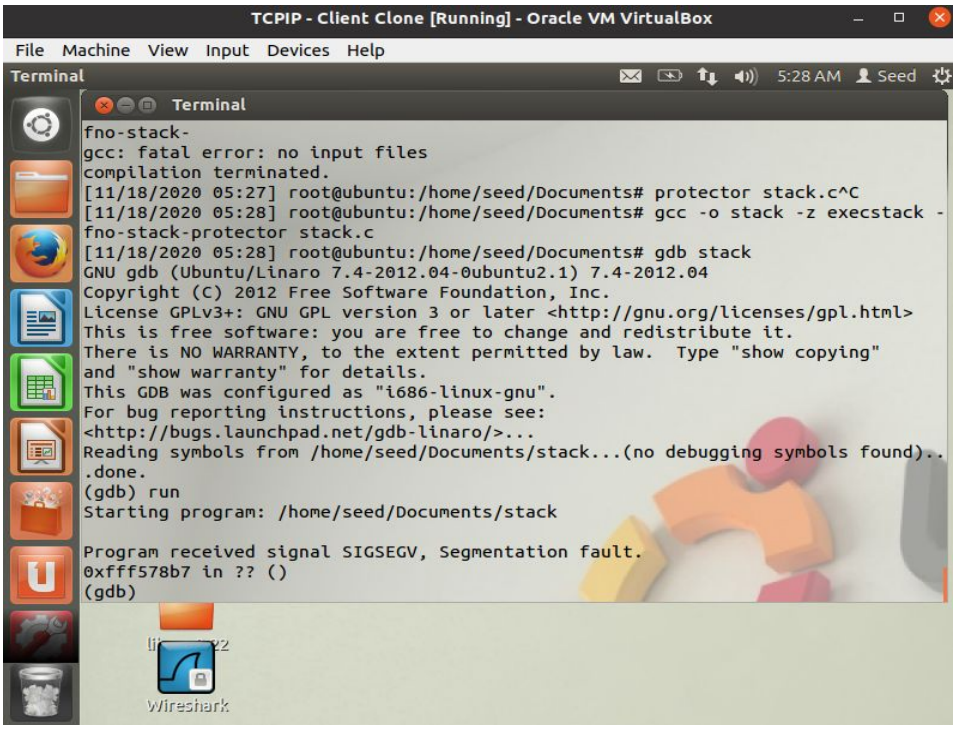
for i in {1..10000}; do echo "12345678912345678901234 5" >> badfile ; done



Percebemos que deu segmentação de memória na execução do programa.

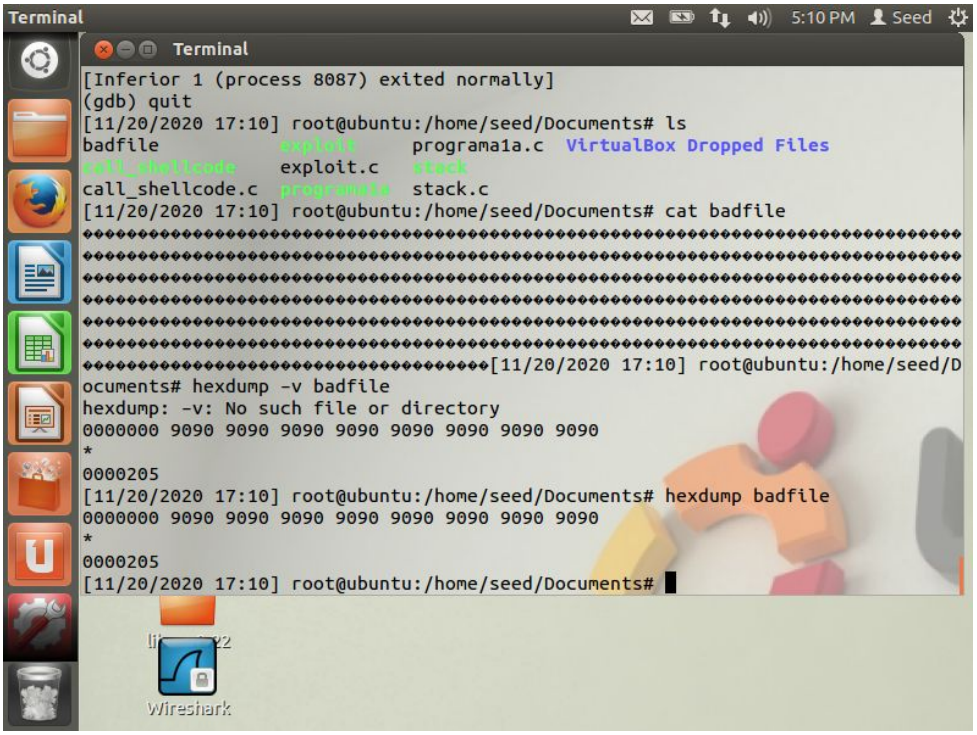
e)

Compilando como root, o resultado também foi de segmentação de memória.

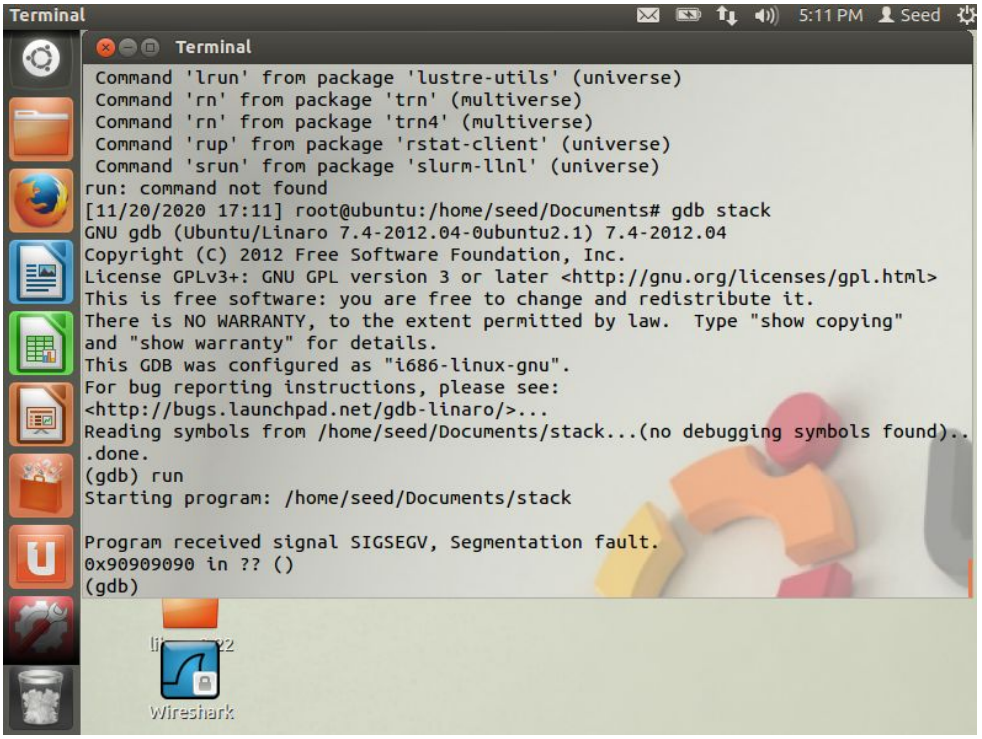


4)

O Shell code foi gravado no arquivo badfile:

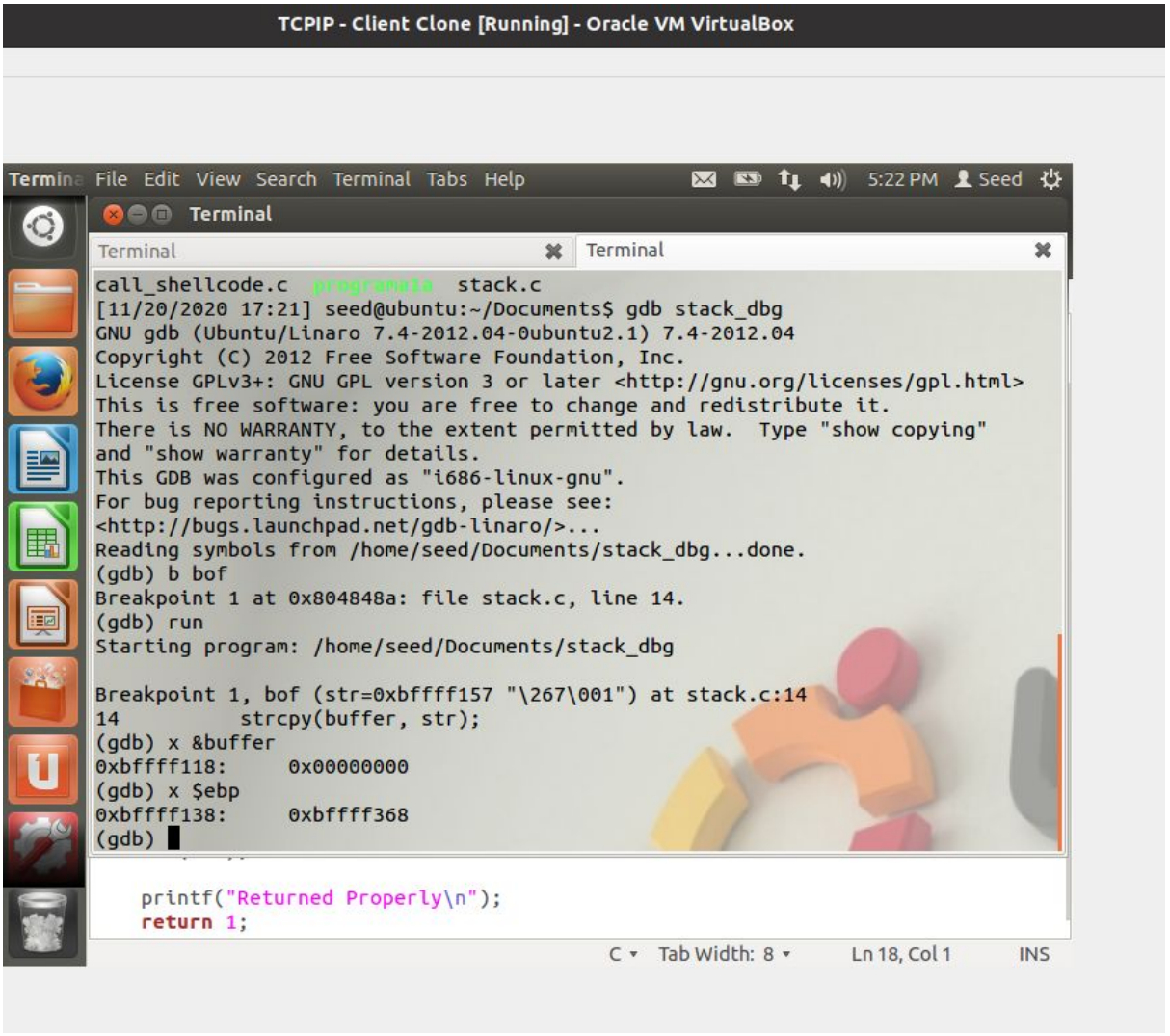


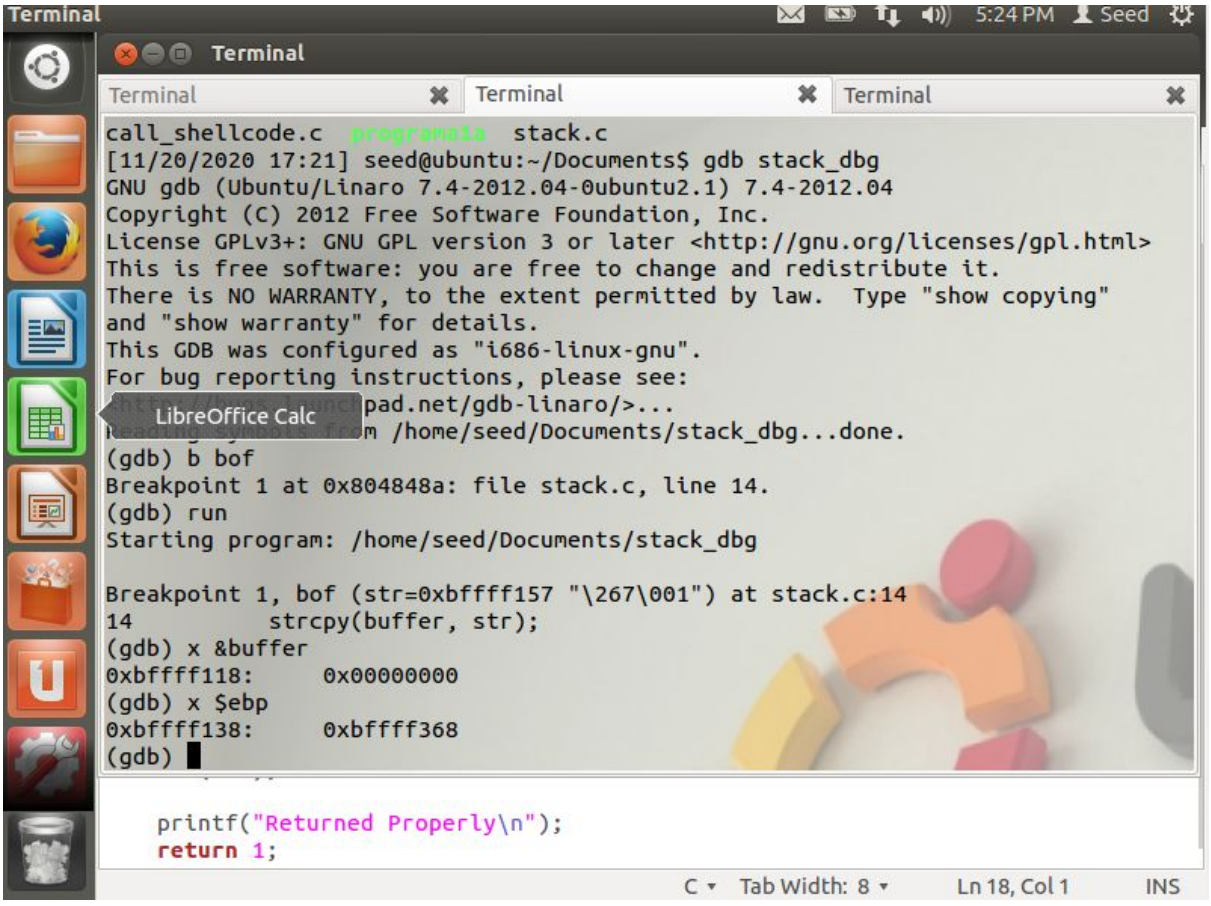
Porém, ao executar o programa stack.c, deu Segmentation Fault:

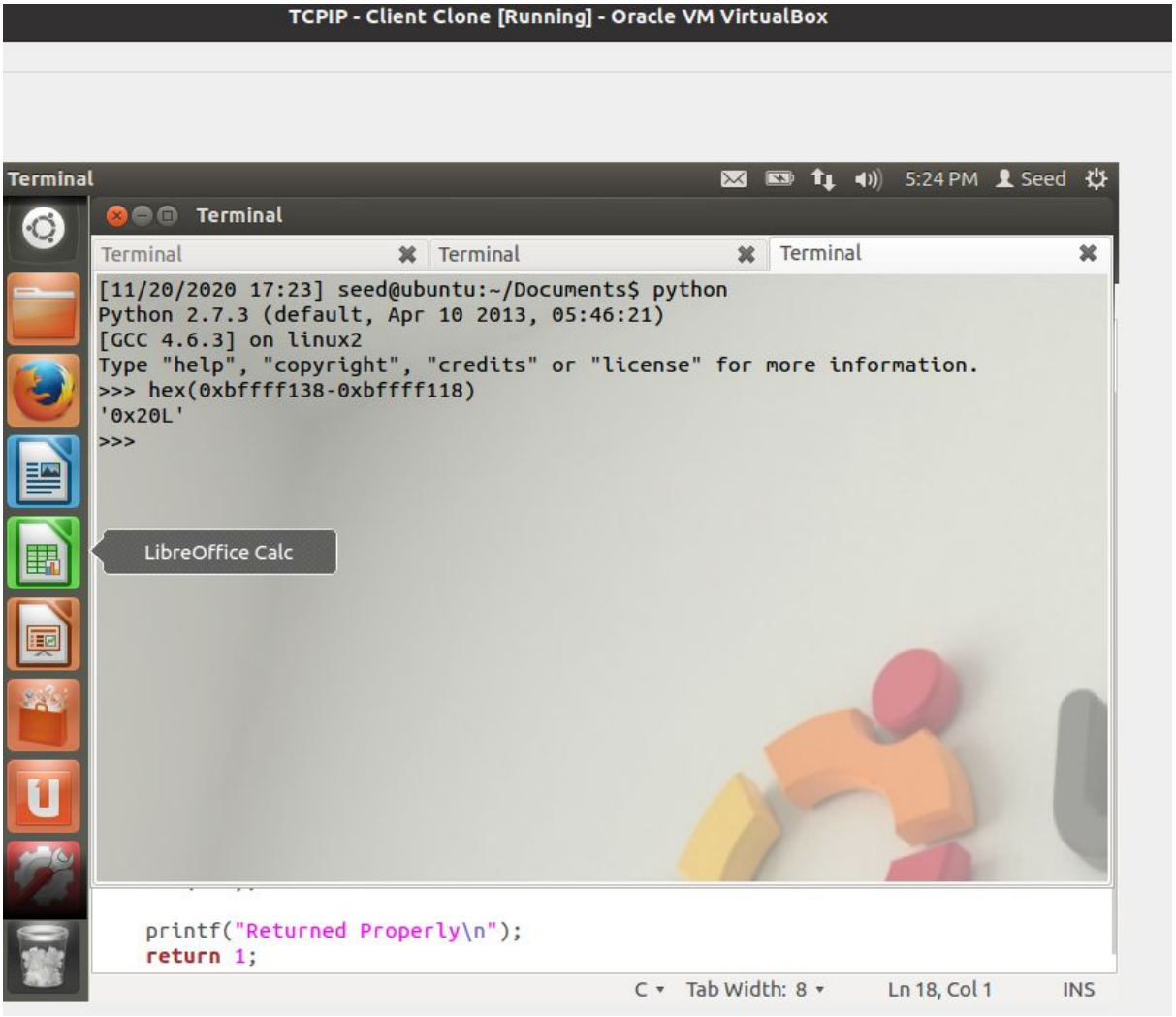


5-

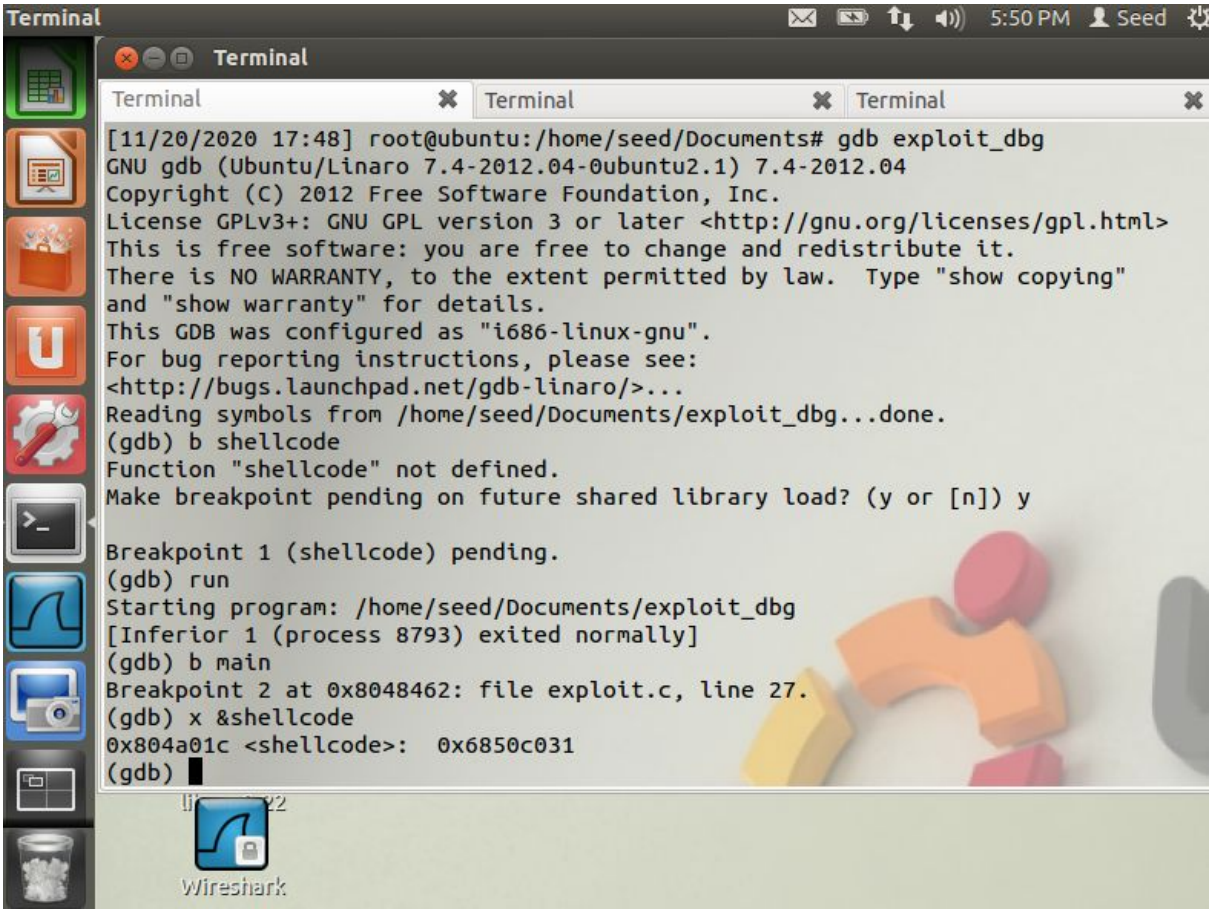
Encontrando o valor do endereço de memória e o valor do endereço de memória do ponteiro:



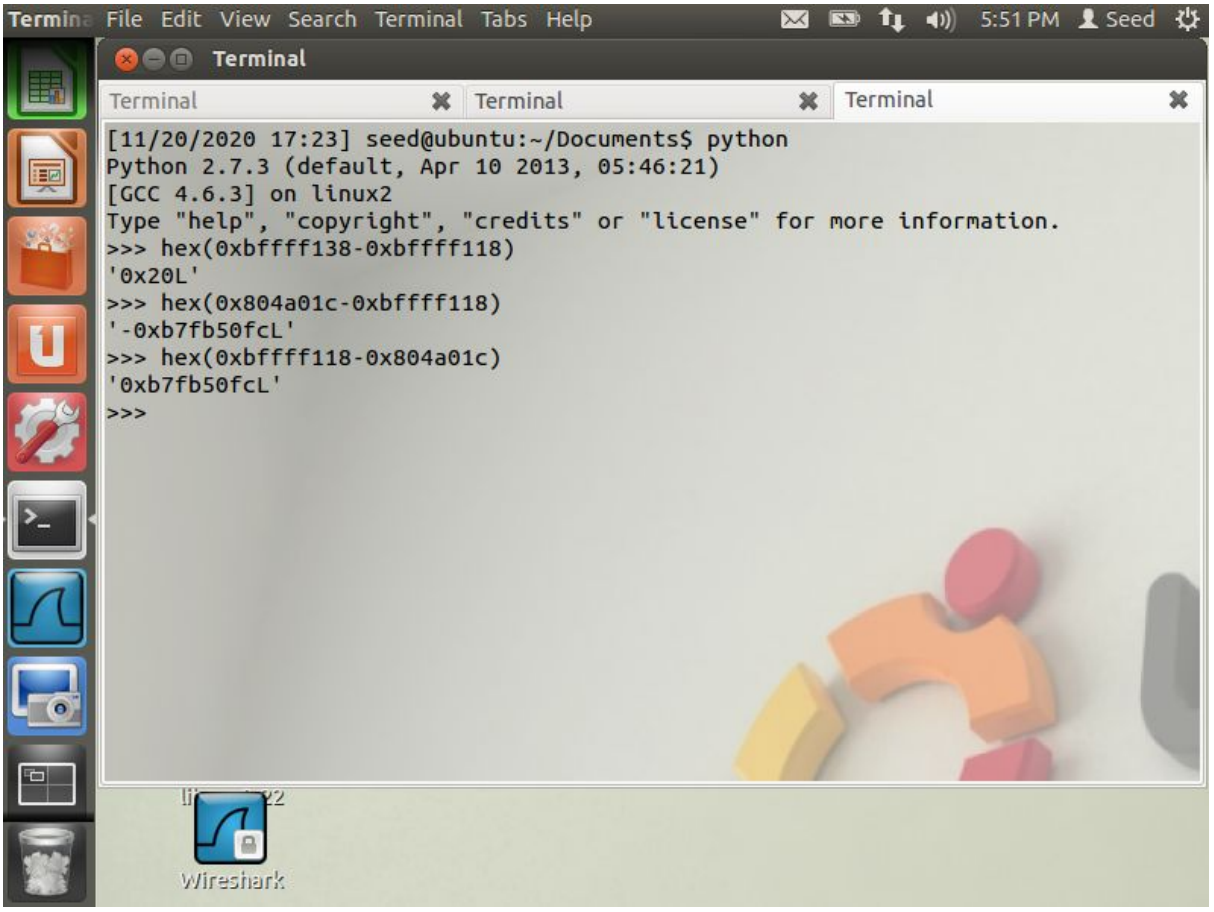

Terminal
5:24 PM
Seed
Terminal
Terminal
Terminal
Terminal
call_shellcode.c programas stack.c
[11/20/2020 17:21] seed@ubuntu:~/Documents$ gdb stack_dbg
GNU gdb (Ubuntu/Linaro 7.4-2012.04-0ubuntu2.1) 7.4-2012.04
Copyright (C) 2012 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
For bug reporting instructions, please see:
pad.net/gdb-linaro/>...
m /home/seed/Documents/stack_dbg...done.
LibreOffice Calc
(gdb) b bof
Breakpoint 1 at 0x804848a: file stack.c, line 14.
(gdb) run
Starting program: /home/seed/Documents/stack_dbg
Breakpoint 1, bof (str=0xbffff157 "\267\001") at stack.c:14
14 strcpy(buffer, str);
(gdb) x &buffer
0xbffff118: 0x00000000
(gdb) x $ebp
0xbffff138: 0xbffff368
(gdb)
printf("Returned Properly\n");
return 1;
C
Tab Width: 8
Ln 18, Col 1
INS



endereço shell code:



distância entre o shellcode e o buffer:


Terminal File Edit View Search Terminal Tabs Help
5:51 PM Seed
Terminal
Terminal
Terminal
Terminal
[11/20/2020 17:23] seed@ubuntu:~/Documents$ python
Python 2.7.3 (default, Apr 10 2013, 05:46:21)
[GCC 4.6.3] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> hex(0xbffff138-0xbffff118)
'0x20L'
>>> hex(0x804a01c-0xbffff118)
'-0xb7fb50fcL'
>>> hex(0xbffff118-0x804a01c)
'0xb7fb50fcL'
>>>
Wireshark